

PROJETO ROTA VITAL

Sistema de Gestão Logística de Hemocomponentes

CESAR School · Análise e Desenvolvimento de Sistemas

Disciplina: Infraestrutura de Software (2026.2)

Trabalho U2 — Parte 2: Planejamento da Remodelagem Arquitetural

Repositório: github.com/Lucas-Alves123/PI2

Integrante	Papel no Squad
Gabriel Farias	Engenharia de Software
Gabriel Nogueira	Infraestrutura e Banco de Dados
Lucas Mateus (@Lucas-Alves123)	Arquiteto de Software & Full-Stack
Mirella Emily	Qualidade de Software & BDD
Myllena Cristiane	Análise de Requisitos & Domínio

Domínio — Rota Vital

Projeto: Rota Vital — Sistema de Gestão Logística de Hemocomponentes
Disciplina: Infraestrutura de Software · CESAR School · ADS · 2026.2
Trabalho: U2 · Parte 2 · Seção A — Remodelagem do Domínio com DDD

O que é o Rota Vital

O Rota Vital é um sistema de gestão logística de hemocomponentes. Ele organiza o controle de estoque de bolsas de sangue, o atendimento de solicitações hospitalares, a alocação de bolsas compatíveis e o acompanhamento da expedição e do transporte em cadeia fria.

O sistema permite que hospitais registrem solicitações de hemocomponentes, que o estoque seja consultado e que todo o processo — desde a separação até a entrega — seja acompanhado com rastreabilidade.

Todos os dados são sintéticos e fictícios.

Subdomínios

O domínio do Rota Vital foi dividido em sete subdomínios, classificados em três categorias: Core, Suporte e Genérico.

Core — Subdomínios que diferenciam o Rota Vital

São os subdomínios que contêm as regras de negócio exclusivas e estratégicas do sistema. Sem eles, o Rota Vital não existe.

1. Requisição e Alocação

Responsabilidade: Receber solicitações de hemocomponentes dos hospitais, verificar a compatibilidade ABO/Rh, aplicar a regra de prioridade FEFO (First Expired, First Out) e alocar as bolsas disponíveis para cada requisição.

Justificativa: É o coração do sistema. Nenhum outro software de logística de hemocomponentes toma as mesmas decisões — compatibilidade sanguínea, FEFO e controle de reserva simultânea são regras exclusivas deste domínio.

---

2. Estoque de Hemocomponentes

Responsabilidade: Controlar o ciclo de vida das bolsas de sangue — entrada, validade, localização, disponibilidade e descarte. Garantir que bolsas vencidas não sejam alocadas.

Justificativa: O controle de estoque de hemocomponentes tem regras próprias: vencimento crítico, rastreabilidade por bolsa individual e restrições de uso. Não é um estoque genérico.

---

3. Expedição e Cadeia Fria

Responsabilidade: Organizar guias de expedição, registrar rotas, acompanhar entregas e monitorar as condições de temperatura durante o transporte.

Justificativa: A cadeia fria é requisito regulatório no transporte de hemocomponentes. A lógica de emissão de guias, controle de temperatura e registro de entrega é específica deste domínio.

Suporte — Subdomínios que apoiam os contextos core

São importantes para o funcionamento do sistema, mas não são o diferencial competitivo.

4. Hospitais

Responsabilidade: Cadastrar e manter informações dos hospitais e unidades solicitantes. Fornecer dados de identidade para os outros contextos.

Justificativa: Sem o cadastro de hospitais, nenhuma solicitação pode ser feita. Mas as regras de cadastro são simples e não exigem lógica especializada.

---

5. Telemetria

Responsabilidade: Registrar leituras de temperatura, localização GPS e dados de veículos durante o transporte. Gerar alertas quando os valores saem da faixa configurada.

Justificativa: Apoia o contexto de Expedição e Cadeia Fria, mas pode ser separado fisicamente em um serviço independente — especialmente em cenários de alto volume, onde o número de leituras cresce muito mais rápido do que o número de requisições.

Genérico — Subdomínios que podem ser comprados ou reutilizados

São subdomínios cujas soluções já existem no mercado e não precisam ser desenvolvidas do zero.

6. Notificação

Responsabilidade: Enviar alertas e notificações internas — nova solicitação, bolsa próxima do vencimento, expedição iniciada, entrega concluída, temperatura fora dos limites.

Justificativa: O mecanismo de envio de notificações (e-mail, SMS, push) é genérico. O que é específico do Rota Vital é *quando* notificar — e essa lógica pertence aos outros contextos.

---

7. Identidade e Acesso

Responsabilidade: Gerenciar usuários, autenticação, perfis e permissões. Controlar quem pode acessar cada parte do sistema.

Justificativa: Autenticação e autorização são resolvidos por soluções de mercado (Spring Security, OAuth2, Keycloak). Não há lógica de negócio exclusiva aqui.

Bounded Contexts

Cada subdomínio é implementado como um Bounded Context independente, com seu próprio modelo de domínio e linguagem ubíqua.

#	Bounded Context	Tipo	Responsabilidade em uma frase
1	**Requisição**	Core	Recebe e gerencia solicitações de hemocomponentes dos hospitais
2	**Alocação**	Core	Verifica compatibilidade, aplica FEFO e reserva bolsas para cada requisição
3	**Estoque**	Core	Controla o ciclo de vida das bolsas desde a entrada até o descarte
4	**Expedição**	Core	Emite guias, organiza o transporte e registra a entrega
5	**Hospitais**	Suporte	Cadastra e mantém hospitais e unidades solicitantes
6	**Telemetria**	Suporte	Registra leituras de temperatura e localização durante o transporte
7	**Notificação**	Genérico	Envia alertas e avisos para os usuários do sistema
8	**Identidade**	Genérico	Gerencia usuários, autenticação e permissões de acesso

> **Nota:** Os contextos de Requisição e Alocação são separados na classificação de responsabilidade, mas na implementação atual (monolito modular) compartilham o mesmo módulo. A separação física em serviços independentes é descrita na projeção de microsserviços.

Referências

- EVANS, E. *Domain-Driven Design*. Addison-Wesley, 2003.
- VERNON, V. *Implementing Domain-Driven Design*. Addison-Wesley, 2013.
- Slides das aulas 006 e 007 — Infraestrutura de Software · CESAR School · 2026.2

Glossário — Linguagem Ubíqua do Rota Vital

Projeto: Rota Vital
Disciplina: Infraestrutura de Software · CESAR School · ADS · 2026.2
Trabalho: U2 · Parte 2 · Seção A — Glossário

Introdução

A linguagem ubíqua é o vocabulário compartilhado entre desenvolvedores e especialistas do domínio. No Rota Vital, ela garante que termos como "bolsa" e "requisição" signifiquem a mesma coisa para todos — ou que, quando um termo tem significados diferentes em contextos distintos, essa diferença seja explicitamente documentada.

Atenção: Alguns termos têm significados diferentes dependendo do contexto. Esses casos são identificados com o símbolo ⚠.

Termos Gerais

Hospital

Definição: Instituição de saúde cadastrada no Rota Vital que pode solicitar hemocomponentes para atender suas necessidades clínicas.
Contexto: Hospitais
Atributos relevantes: nome, CNPJ (fictício), endereço, cidade, estado, responsável, status (ativo/inativo).
Usado em: Hospitais, Requisição

Unidade Solicitante

Definição: Setor ou unidade interna de um hospital habilitada a realizar solicitações de hemocomponentes. Exemplos: UTI, Pronto-Socorro, Bloco Cirúrgico.
Contexto: Hospitais
Nota: Um hospital pode ter várias unidades solicitantes. A unidade identifica a origem da requisição dentro do hospital.
Usado em: Hospitais, Requisição

Requisição

Definição: Solicitação formal registrada por um hospital para receber uma quantidade específica de hemocomponentes com determinado tipo sanguíneo e nível de urgência.
Contexto: Requisição
Ciclo de status: Pendente → Em análise → Parcialmente atendida → Atendida → Em expedição → Concluída / Cancelada
Usado em: Requisição, Alocação, Expedição

Alocação

Definição: Ato de reservar uma bolsa compatível para uma requisição específica. Uma alocação registra qual bolsa foi reservada, quando e para qual requisição.
Contexto: Alocação
Invariante: Uma bolsa só pode estar alocada em uma requisição ativa por vez.
Usado em: Alocação, Estoque

Compatibilidade

Definição: Regra que determina se uma bolsa pode ser transfundida para o receptor considerando os tipos sanguíneos ABO e Rh. A compatibilidade varia conforme o componente (concentrado de hemácias, plaquetas, plasma, etc.).
Contexto: Alocação
Nota: A matriz de compatibilidade utilizada neste sistema é sintética e deve ser validada por profissional habilitado antes de qualquer uso real.
Usado em: Alocação

Tipo Sanguíneo

Definição: Classificação do sangue segundo os sistemas ABO (A, B, AB, O) e Rh (positivo ou negativo). Exemplo: "O negativo", "AB positivo".
Contexto: Alocação, Estoque
Implementação: Value Object com atributos `abo` (enum: A, B, AB, O) e `rh` (enum: POSITIVO, NEGATIVO).
Usado em: Alocação, Estoque, Requisição

FEFO (First Expired, First Out)

Definição: Regra de priorização que determina que, entre as bolsas compatíveis disponíveis, a que tem a data de validade mais próxima deve ser alocada primeiro. Minimiza o descarte por vencimento.
Contexto: Alocação
Analogia: Similar ao FIFO (First In, First Out) dos estoques convencionais, mas baseado em validade, não em ordem de entrada.
Usado em: Alocação

⚠ Bolsa (Estoque)

Definição no contexto Estoque: Item individual armazenado no banco de sangue. Identificada por um código único, possui tipo sanguíneo, tipo de hemocomponente, data de coleta, data de validade, localização física e status (Disponível, Reservada, Em separação, Em expedição, Entregue, Descartada, Vencida).
Contexto: Estoque
Regra: Bolsas vencidas não podem ser alocadas. O sistema verifica a validade automaticamente.

⚠ Bolsa (Expedição)

Definição no contexto Expedição: Volume transportado vinculado a uma guia de expedição. Neste contexto, o que importa não é o conteúdo interno da bolsa, mas sua associação com a guia, o veículo e as condições de temperatura registradas durante o transporte.

Contexto: Expedição
Diferença crítica: No Estoque, "bolsa" é um item de estoque com validade e localização. Na Expedição, "bolsa" é um volume em trânsito, monitorado pela cadeia fria.

> ⚠ **Atenção:** "Bolsa" tem significados diferentes em Estoque e Expedição. Os dois modelos não devem ser misturados. Um controller de Expedição não deve acessar diretamente o repositório de bolsas do Estoque.

Hemocomponente

Definição: Produto derivado do processamento do sangue total. Os principais tipos são: Concentrado de Hemácias (CH), Plasma Fresco Congelado (PFC), Concentrado de Plaquetas (CP) e Crioprecipitado.
Contexto: Estoque, Requisição
Nota: A compatibilidade ABO/Rh varia conforme o hemocomponente.
Usado em: Estoque, Requisição, Alocação

Guia de Expedição

Definição: Documento (ou registro digital) que formaliza o envio de bolsas de um banco de sangue para um hospital. Contém as bolsas embarcadas, o veículo, o responsável, a data/hora de saída e o destino.
Contexto: Expedição
Usado em: Expedição, Telemetria

Cadeia Fria

Definição: Conjunto de procedimentos e controles que garantem que os hemocomponentes sejam mantidos dentro da faixa de temperatura adequada desde a coleta até a entrega no hospital.
Contexto: Expedição, Telemetria
Nota: Os limites de temperatura são configuráveis no sistema e devem ser definidos pela equipe com base em premissas do domínio. Os valores padrão são sintéticos.
Usado em: Expedição, Telemetria

Telemetria

Definição: Registro automático de dados do transporte — temperatura, localização GPS, identificador do veículo e data/hora da leitura. Permite monitorar se a cadeia fria está sendo mantida.
Contexto: Telemetria
Usado em: Telemetria, Expedição

Alerta de Temperatura

Definição: Notificação gerada automaticamente quando uma leitura de telemetria indica temperatura fora da faixa configurada para a cadeia fria.
Contexto: Telemetria, Notificação
Usado em: Telemetria, Notificação

Resumo — Termos com Duplo Significado

Termo	Contexto A	Significado A	Contexto B	Significado B
Bolsa	Estoque	Item armazenado com validade, localização e status	Expedição	Volume em trânsito vinculado a uma guia e monitorado por temperatura

---

Referências

- EVANS, E. *Domain-Driven Design*. Addison-Wesley, 2003.
- VERNON, V. *Implementing Domain-Driven Design*. Addison-Wesley, 2013.

Mapa de Contextos — Rota Vital

Projeto: Rota Vital
Disciplina: Infraestrutura de Software · CESAR School · ADS · 2026.2
Trabalho: U2 · Parte 2 · Seção A — Mapa de Contextos

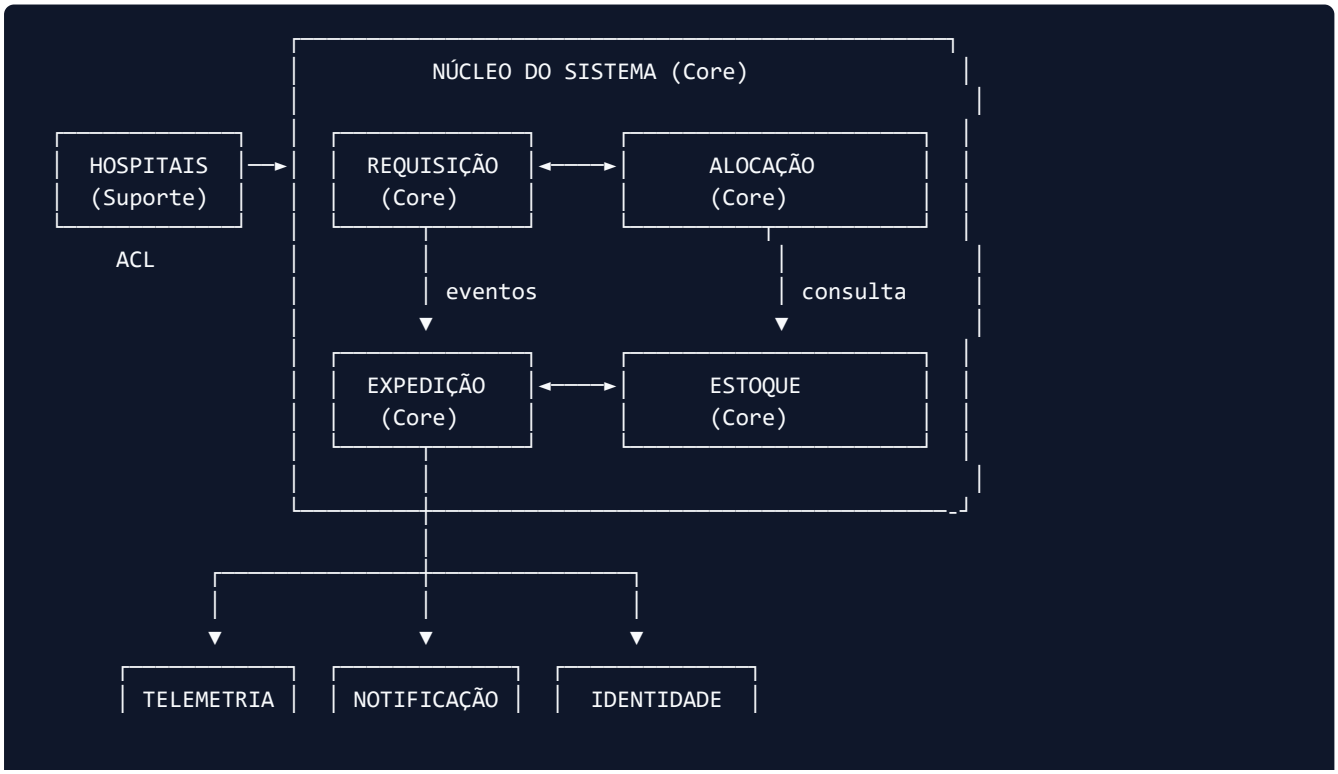
Introdução

O mapa de contextos descreve como os Bounded Contexts do Rota Vital se relacionam entre si. Cada relação tem um tipo que define quem depende de quem, como os dados trafegam e quem tem autonomia para mudar o modelo.

Tipos de relação utilizados

Tipo	Descrição
Cliente-Fornecedor (C/F)	O contexto upstream (fornecedor) provê dados; o downstream (cliente) os consome. O fornecedor tem autonomia sobre seu modelo.
Conformista (C)	O contexto downstream adota o modelo do upstream sem adaptação.
Camada Anticorrupção (ACL)	O contexto downstream traduz o modelo do upstream para seu próprio modelo, protegendo sua linguagem ubíqua.
Publicador-Assinante (Pub/Sub)	O contexto publica eventos de domínio; outros contextos assinam e reagem a eles.

Diagrama do Mapa de Contextos



```
        │ (Suporte)    │   │ (Genérico)   │   │ (Genérico)   │
        └──────────────┘   └──────────────┘   └──────────────┘
```

Relações Detalhadas

1. Hospitais → Requisição

Tipo: Cliente-Fornecedor com Camada Anticorrupção (ACL)
Descrição: O contexto de Requisição precisa saber qual hospital fez a solicitação e qual unidade solicitante. Ele consulta o contexto de Hospitais para obter essas informações, mas traduz o modelo para seu próprio vocabulário via ACL.
O que trafega: `hospitalId`, `unidadeId` — apenas identificadores, não o objeto completo.
Motivo da ACL: Requisição não deve depender do modelo interno de Hospital. Se Hospital mudar, a ACL absorve a mudança.

---

2. Requisição → Alocação

Tipo: Cliente-Fornecedor (Requisição é downstream, Alocação é upstream)
Descrição: Quando uma requisição é registrada, o contexto de Alocação é acionado para verificar compatibilidade, aplicar FEFO e reservar bolsas. O resultado retorna como lista de alocações vinculadas à requisição.
O que trafega: `tipoSanguineo`, `componente`, `quantidade`, `urgencia` → retorna `Alocacao[]`
Nota: Na implementação monolítica atual, esta comunicação é feita por chamada direta de serviço. Na projeção de microsserviços, seria via evento (`RequisicaoRegistrada`).

---

3. Alocação → Estoque

Tipo: Cliente-Fornecedor com Camada Anticorrupção
Descrição: O contexto de Alocação consulta o Estoque para identificar bolsas disponíveis, verificar compatibilidade e marcar bolsas como reservadas. O Estoque tem autonomia sobre seu modelo interno de bolsa.
O que trafega: consulta por tipo sanguíneo + componente + disponibilidade; comando de reserva (`bolsaId`)
Invariante protegida: O Estoque é responsável por garantir que uma bolsa só possa ser reservada por uma requisição ativa.

---

4. Requisição → Expedição (evento)

Tipo: Publicador-Assinante
Descrição: Quando uma requisição atinge o status "Atendida" (todas as alocações concluídas), ela publica o evento `RequisicaoAtendida`. O contexto de Expedição assina esse evento e prepara a guia de expedição.
Evento publicado: `RequisicaoAtendida { requisicaoId, hospitalId, bolsasAlocadas[] }`
Nota: Na implementação monolítica, este evento é tratado internamente (listener Spring). Na projeção de microsserviços, seria via fila (RabbitMQ / SQS).

---

5. Expedição → Telemetria

Tipo: Cliente-Fornecedor
Descrição: O contexto de Telemetria recebe o identificador da guia de expedição e registra leituras de temperatura e localização vinculadas a ela. A Expedição consulta a Telemetria para obter o histórico de leituras.
O que trafega: `guiaId`, leituras (`temperatura`, `latitude`, `longitude`, `timestamp`)

---

6. Telemetria → Notificação (evento)

Tipo: Publicador-Assinante
Descrição: Quando uma leitura de temperatura fica fora da faixa configurada, a Telemetria publica o evento `AlertaTemperatura`. O contexto de Notificação assina e gera o alerta.
Evento publicado: `AlertaTemperatura { guiaId, temperatura, timestamp, limiteMinimo, limiteMaximo }`

---

7. Requisição → Notificação (evento)

Tipo: Publicador-Assinante
Descrição: A Requisição publica eventos (`RequisicaoRegistrada`, `RequisicaoAtendida`) que a Notificação assina para gerar avisos para os usuários.

---

8. Estoque → Notificação (evento)

Tipo: Publicador-Assinante
Descrição: O Estoque publica `BolsaProximaDoVencimento` e `BolsaVencida`. A Notificação assina e alerta os operadores.

---

9. Todos os contextos → Identidade

Tipo: Conformista
Descrição: Todos os contextos delegam autenticação e autorização ao contexto de Identidade. Os contextos consomem os dados do usuário autenticado (id, perfil, permissões) sem tradução — conformando ao modelo da Identidade.

Eventos de Domínio (opcional — entre contextos)

Evento	Publicado por	Consumido por	Descrição
`RequisicaoRegistrada`	Requisição	Notificação	Nova solicitação recebida
`BolsaReservada`	Alocação	Estoque, Dashboard	Bolsa alocada para uma requisição
`RequisicaoAtendida`	Requisição	Expedição, Notificação	Todas as alocações concluídas
`GuiaEmitida`	Expedição	Telemetria, Notificação	Expedição iniciada
`EntregaConcluida`	Expedição	Requisição, Notificação	Hemocomponentes entregues no hospital
`AlertaTemperatura`	Telemetria	Notificação	Temperatura fora da faixa configurada
`BolsaProximaDoVencimento`	Estoque	Notificação	Bolsa com validade próxima

Evento	Publicado por	Consumido por	Descrição
`BolsaVencida`	Estoque	Notificação, Alocação	Bolsa vencida — deve ser bloqueada

---

Referências

- EVANS, E. *Domain-Driven Design*. Addison-Wesley, 2003.
- VERNON, V. *Implementing Domain-Driven Design*. Addison-Wesley, 2013.
- Slides das aulas 006 e 007 — Infraestrutura de Software · CESAR School · 2026.2

Modelo Tático — Bounded Context: Requisição e Alocação

Projeto: Rota Vital
Disciplina: Infraestrutura de Software · CESAR School · ADS · 2026.2
Trabalho: U2 · Parte 2 · Seção B — Modelo Tático de um Contexto Core

---

Contexto escolhido: Requisição e Alocação

Este é o Bounded Context principal (Core) do Rota Vital. Ele concentra as regras de negócio mais críticas: receber solicitações de hemocomponentes, verificar compatibilidade ABO/Rh, aplicar a regra FEFO e reservar bolsas de maneira segura e exclusiva.

Persona

Nome fictício: Ana Beatriz Costa
Papel: Atendente de Hemoterapia — responsável por registrar solicitações hospitalares no Rota Vital
Contexto: Ana trabalha no banco de sangue. Quando um hospital liga pedindo hemocomponentes, ela acessa o sistema, informa o hospital, o tipo sanguíneo, o componente e a urgência. O sistema faz o restante: verifica estoque, checa compatibilidade, aplica FEFO e reserva as bolsas.
Frustração atual (sem sistema): Sem o Rota Vital, Ana consulta planilhas, verifica o estoque manualmente e corre o risco de reservar a mesma bolsa para dois hospitais ao mesmo tempo.

História de Usuário Âncora

> **Como** atendente de hemoterapia responsável pelo atendimento de um hospital,
> **Quero** solicitar bolsas de sangue informando o tipo sanguíneo e o componente necessário,
> **Para que** o sistema selecione e reserve automaticamente as bolsas compatíveis disponíveis, priorizando as que vencem primeiro.

Critérios de Aceite

Os critérios abaixo são testáveis e estão vinculados às invariantes do agregado.

CA1 — Rejeitar bolsas incompatíveis
Dado que existe uma bolsa do tipo AB+ disponível,
quando o sistema processar uma requisição solicitando O-,
então a bolsa AB+ não deve ser alocada para essa requisição.

CA2 — Priorizar a validade mais próxima (FEFO)
Dado que existem duas bolsas O+ disponíveis com datas de validade diferentes,
quando o sistema alocar bolsas para uma requisição O+,
então a bolsa com vencimento mais próximo deve ser alocada primeiro.

CA3 — Impedir reserva duplicada
Dado que uma bolsa já está reservada para a Requisição #001,
quando o sistema tentar alocar a mesma bolsa para a Requisição #002,
então o sistema deve rejeitar a operação e manter a bolsa vinculada apenas à Requisição #001.

CA4 — Concluir requisição somente com todas as alocações realizadas
Dado que uma requisição solicita 3 bolsas e apenas 2 foram alocadas,
quando o sistema verificar o status da requisição,
então o status deve permanecer como "Parcialmente atendida" e não "Atendida".

Agregado: Requisição

Raiz do Agregado — `Requisicao`

A `Requisicao` é a raiz do agregado. Ela controla todo o ciclo de vida da solicitação e garante que as invariantes sejam respeitadas.

```
// Representação conceitual (não é código de produção final)
public class Requisicao {
    private UUID id;
    private UUID hospitalId;           // referência ao contexto Hospitais
    private UUID unidadeId;            // referência à unidade solicitante
    private TipoSanguineo tipoSanguineo; // Value Object
    private Componente componente;     // enum: CH, PFC, CP, CRIO
    private int quantidadeSolicitada;
    private Urgencia urgencia;         // enum: ROTINA, URGENCIA, EMERGENCIA
    private StatusRequisicao status;
    private LocalDateTime criadaEm;
    private List<Alocacao> alocacoes;  // entidades internas
}
```

Invariantes protegidas pela raiz: - Não permite adicionar uma alocação com bolsa já alocada em outra requisição ativa. - Não permite mudar o status para ATENDIDA se `alocacoes.size() < quantidadeSolicitada`. - Não permite cancelar uma requisição que já está em expedição.

Entidade Interna — `Alocacao`

A `Alocacao` representa a reserva de uma bolsa específica para esta requisição. Ela não tem existência fora do agregado `Requisicao`.

```
// Representação conceitual
public class Alocacao {
    private UUID id;
    private UUID bolsaId;              // referência ao contexto Estoque
    private Componente componente;
    private LocalDate dataVencimentoBolsa; // copiado no momento da alocação (FEFO)
    private LocalDateTime dataAlocacao;
    private StatusAlocacao status;     // enum: RESERVADA, CONFIRMADA, CANCELADA
}
```

Regra: `bolsaId` é uma referência ao contexto de Estoque — não é um objeto `Bolsa` importado. Os contextos não compartilham classes de domínio.

Value Object — `TipoSanguineo`

Imutável, sem identidade própria. Dois objetos com o mesmo ABO e Rh são considerados iguais.

```
// Representação conceitual
public final class TipoSanguineo {
    private final ABO abo;      // enum: A, B, AB, O
    private final Rh rh;        // enum: POSITIVO, NEGATIVO

    // Construtor valida que abo e rh não são nulos
    // equals e hashCode baseados em abo + rh
    // toString: "O-", "AB+", "A+"

    public boolean ehCompativel(TipoSanguineo receptor, Componente componente) {
        // Consulta a MatrizCompatibilidade (configurável)
        // Retorna true se este tipo pode ser doado para o receptor
    }
}
```

Nota sobre a matriz de compatibilidade:
A tabela abaixo é **sintética e deve ser validada por profissional habilitado** antes de qualquer uso real. É uma premissa do projeto para fins acadêmicos.

Tipo do doador	Pode ser recebido por (Concentrado de Hemácias)
O-	O-, O+, A-, A+, B-, B+, AB-, AB+
O+	O+, A+, B+, AB+
A-	A-, A+, AB-, AB+
A+	A+, AB+
B-	B-, B+, AB-, AB+
B+	B+, AB+
AB-	AB-, AB+
AB+	AB+

> ⚠️ **Premissa não validada:** Esta matriz representa a compatibilidade para Concentrado de Hemácias. Para outros componentes (plaquetas, plasma), as regras diferem. A `MatrizCompatibilidade` deve ser configurável no sistema.

Invariantes do Agregado

#	Invariante	Onde é verificada
1	Só bolsas compatíveis com o tipo sanguíneo solicitado podem ser alocadas	`Requisicao.alocar()`
2	Entre as bolsas compatíveis, priorizar a que vence primeiro (FEFO)	`ServicoAlocacao.selecionarBolsas()`
3	Uma bolsa não pode estar reservada em duas requisições ativas simultaneamente	`BolsaRepository` + constraint de banco
4	Uma requisição só pode ter status ATENDIDA quando todas as alocações estão confirmadas	`Requisicao.concluir()`
5	Bolsas vencidas não podem ser alocadas	`Estoque` (verificado antes de retornar bolsas disponíveis)

Enumerações

```
enum Componente {
    CH,   // Concentrado de Hemácias
    PFC,  // Plasma Fresco Congelado
    CP,   // Concentrado de Plaquetas
    CRIO  // Crioprecipitado
}

enum Urgencia {
    ROTINA,
    URGENCIA,
    EMERGENCIA
}

enum StatusRequisicao {
    PENDENTE,
    EM_ANALISE,
    PARCIALMENTE_ATENDIDA,
    ATENDIDA,
    EM_EXPEDICAO,
    CONCLUIDA,
    CANCELADA
}

enum StatusAlocacao {
    RESERVADA,
    CONFIRMADA,
    CANCELADA
}
```

Referências

- EVANS, E. *Domain-Driven Design*. Addison-Wesley, 2003.
- VERNON, V. *Implementing Domain-Driven Design*. Addison-Wesley, 2013.
- Slides das aulas 006 e 007 — Infraestrutura de Software · CESAR School · 2026.2

Cenários BDD — Requisição e Alocação

Projeto: Rota Vital
Disciplina: Infraestrutura de Software · CESAR School · ADS · 2026.2
Trabalho: U2 · Parte 2 · Seção B — Cenário Gherkin

Funcionalidade: Alocar bolsas para uma requisição hospitalar

```
# language: pt
Funcionalidade: Alocar bolsas compatíveis para uma requisição hospitalar
  Como atendente de hemoterapia responsável pelo atendimento de um hospital
  Quero solicitar bolsas de sangue informando o tipo sanguíneo e o componente necessário
  Para que o sistema selecione e reserve automaticamente as bolsas compatíveis disponíveis
       priorizando as que vencem primeiro

  Contexto:
    Dado que o hospital "Hospital São Lucas" está cadastrado no sistema
    E a usuária "Ana Beatriz" está autenticada com o perfil "Atendimento"

  # CA2 — FEFO: priorizar a bolsa com vencimento mais próximo
  Cenário: Alocar a bolsa compatível com vencimento mais próximo (FEFO)
    Dado que existem as seguintes bolsas disponíveis no estoque:
      | bolsaId | tipoSanguineo | componente | dataValidade |
      | BOLSA-A | O+            | CH         | 2026-11-25   |
      | BOLSA-B | O+            | CH         | 2026-11-15   |
      | BOLSA-C | O+            | CH         | 2026-12-01   |
    Quando Ana Beatriz registra uma requisição para "Hospital São Lucas" solicitando:
      | campo              | valor |
      | tipoSanguineo      | O+    |
      | componente         | CH    |
      | quantidade         | 1     |
      | urgencia           | ROTINA|
    Então o sistema deve alocar a bolsa "BOLSA-B" (vencimento 2026-11-15)
    E o status da requisição deve ser "Atendida"
    E a bolsa "BOLSA-B" deve ter status "Reservada" no estoque

  # CA1 — Rejeitar bolsa incompatível
  Cenário: Não alocar bolsa com tipo sanguíneo incompatível
    Dado que existe apenas a bolsa "BOLSA-X" do tipo "AB+" com componente "CH" disponível
    Quando Ana Beatriz registra uma requisição solicitando tipo "O-" componente "CH"
    Então o sistema não deve alocar nenhuma bolsa
    E o status da requisição deve ser "Pendente"
    E o sistema deve informar que não há bolsas compatíveis disponíveis

  # CA3 — Impedir reserva duplicada
  Cenário: Não alocar a mesma bolsa para duas requisições ativas
    Dado que a bolsa "BOLSA-D" do tipo "A+" componente "CH" está reservada para a requisição "REQ-00
    Quando o sistema tentar alocar "BOLSA-D" para uma nova requisição "REQ-002" solicitando "A+" CH
    Então o sistema não deve alocar "BOLSA-D" para "REQ-002"
    E a bolsa "BOLSA-D" deve permanecer vinculada apenas à requisição "REQ-001"

  # CA4 — Requisição parcialmente atendida
  Cenário: Manter status como "Parcialmente atendida" quando não há bolsas suficientes
    Dado que existe apenas 1 bolsa "O+" de componente "CH" disponível no estoque
    Quando Ana Beatriz registra uma requisição solicitando 3 bolsas "O+" de componente "CH"
    Então o sistema deve alocar 1 bolsa
```

```
    E o status da requisição deve ser "Parcialmente atendida"
    E o sistema deve informar que apenas 1 de 3 bolsas foram alocadas

  # Regra de urgência — emergência tem prioridade (cenário adicional)
  Cenário: Requisição de emergência é processada antes de requisições de rotina
    Dado que existe apenas 1 bolsa "B-" de componente "CH" disponível
    E existe uma requisição de "ROTINA" pendente para "B-" CH
    Quando o sistema registra uma nova requisição de "EMERGENCIA" para "B-" CH
    Então o sistema deve priorizar a alocação para a requisição de "EMERGENCIA"
```

Mapeamento Critérios de Aceite ↔ Cenários Gherkin

Critério de Aceite	Cenário Gherkin	Invariante
CA1 — Rejeitar incompatíveis	"Não alocar bolsa com tipo sanguíneo incompatível"	Invariante 1
CA2 — Priorizar FEFO	"Alocar a bolsa compatível com vencimento mais próximo"	Invariante 2
CA3 — Impedir duplicata	"Não alocar a mesma bolsa para duas requisições ativas"	Invariante 3
CA4 — Concluir com todas alocações	"Manter status como Parcialmente atendida..."	Invariante 4

Nota sobre a Implementação (Parte 3)

Estes cenários serão convertidos em testes executáveis com **Cucumber** (mesma ferramenta já usada no Mesa DJ) na Parte 3 do trabalho. Os steps (`@Dado` , `@Quando` , `@Então`) serão implementados em Java, usando Testcontainers para subir o PostgreSQL nos testes de aceitação.

O cenário **principal** para a Parte 3 é o FEFO (CA2) — será o primeiro cenário implementado e executado.

Referências

- EVANS, E. *Domain-Driven Design*. Addison-Wesley, 2003.
- CUCUMBER. *Gherkin Reference*. Disponível em: https://cucumber.io/docs/gherkin/
- Slides das aulas 006 e 007 — Infraestrutura de Software · CESAR School · 2026.2

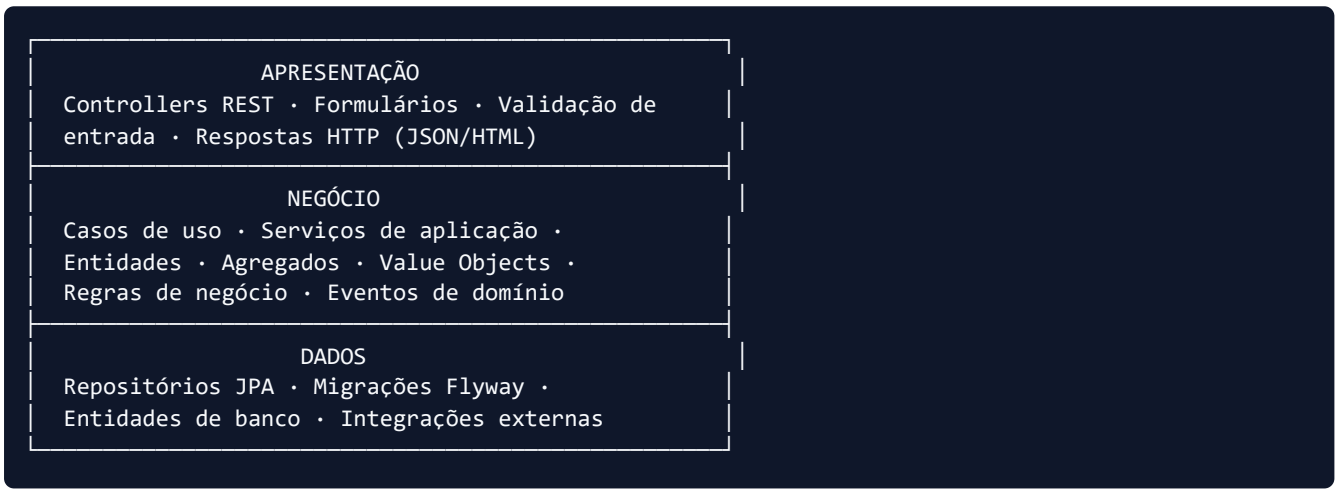
Arquitetura em Três Camadas — Rota Vital

Projeto: Rota Vital
Disciplina: Infraestrutura de Software · CESAR School · ADS · 2026.2
Trabalho: U2 · Parte 2 · Seção C — Arquitetura em Três Camadas

---

Visão Geral

O Rota Vital é organizado como um **monolito modular** com arquitetura em três camadas lógicas. Essa separação garante que cada parte do sistema tenha uma responsabilidade clara e que as regras de negócio fiquem isoladas de detalhes de infraestrutura.



Camada 1: Apresentação

Responsabilidade: Receber requisições HTTP, validar os dados de entrada, chamar a camada de negócio e devolver respostas no formato adequado (JSON para APIs, HTML para interfaces web).

O que fica aqui: - Controllers REST (`@RestController` , `@Controller`) - DTOs de entrada e saída (Request / Response) - Validação de campos (`@Valid` , `@NotNull` , `@NotBlank`) - Tratamento de erros HTTP (`@ControllerAdvice`) - Arquivos estáticos (HTML, CSS, JavaScript)

O que NÃO fica aqui: - Regras de negócio - Acesso direto ao banco de dados - Lógica de compatibilidade ABO/Rh ou FEFO

Exemplo no Rota Vital:

```
RequisicaoController  → recebe POST /api/requisicoes
                      → valida os campos obrigatórios
                      → chama RegistrarRequisicaoUseCase
                      → retorna 201 Created com o id da requisição
```

Camada 2: Negócio

Responsabilidade: Conter toda a lógica de negócio do sistema. É o coração do Rota Vital.

A camada de negócio é dividida em duas subcamadas:

2.1. Aplicação (Casos de Uso)

Orquestra o domínio para executar uma operação de negócio completa. Um caso de uso não tem regras de negócio — ele coordena entidades, repositórios e eventos.

O que fica aqui: - Casos de uso / serviços de aplicação (`RegistrarRequisicao` , `AlocarBolsas` , `ConsultarEstoque`) - Orquestração de chamadas entre entidades e repositórios - Publicação de eventos de domínio - Controle de transações (`@Transactional`)

Exemplo:

```
RegistrarRequisicaoUseCase:
  1. Busca hospital pelo hospitalId
  2. Cria o agregado Requisicao
  3. Chama AlocarBolsasUseCase (compatibilidade + FEFO)
  4. Salva via RequisicaoRepository
  5. Publica evento RequisicaoRegistrada
  6. Retorna RequisicaoResponse
```

2.2. Domínio (Entidades, Agregados, Value Objects)

Contém o modelo de domínio puro. As entidades de domínio têm comportamento — não são apenas estruturas de dados (anemic domain model é um antipadrão aqui).

O que fica aqui: - Entidades (`Requisicao` , `Alocacao` , `Bolsa` , `Hospital`) - Agregados (raiz: `Requisicao`) - Value Objects (`TipoSanguineo` , `CNPJ` , `Validade`) - Enumerações de domínio (`StatusRequisicao` , `Componente` , `Urgencia`) - Exceções de negócio (`RequisicaoNaoEncontradaException` , `BolsaIncompativelException`) - Interfaces de repositório (apenas a interface — a implementação fica na camada de Dados) - Eventos de domínio (`RequisicaoRegistrada` , `BolsaReservada`)

Regra fundamental: O domínio **não deve depender** de controllers, banco de dados ou frameworks web. Uma entidade de domínio é Java puro.

Camada 3: Dados

Responsabilidade: Persistir e recuperar dados. Implementa as interfaces de repositório definidas no domínio.

O que fica aqui: - Implementações de repositório JPA (`RequisicaoRepositoryJpa` , `BolsaRepositoryJpa`) - Entidades JPA (podem ser as mesmas do domínio ou classes separadas com `@Entity`) - Migrações de banco de dados (Flyway — `V1__cria_hospitais.sql` , `V2__cria_bolsas.sql` , etc.) - Configuração de conexão com banco (`application.yml`) - Integrações externas (APIs de terceiros, mensageria)

O que NÃO fica aqui: - Regras de negócio - Lógica de compatibilidade - Código de apresentação

---

Regra de Dependência

```
Apresentação  →  Negócio (Aplicação)  →  Negócio (Domínio)
     ↓                   ↓
   Dados              Dados (implementa interfaces do Domínio)
```

Em palavras: - A **Apresentação** usa a camada de **Negócio** (casos de uso) — nunca acessa o banco diretamente. - A camada de **Negócio (Aplicação)** coordena o **Domínio** e usa interfaces de repositório. - O **Domínio** não depende de nada externo — nem de Spring, nem de JPA, nem de HTTP. - A camada de **Dados** implementa as interfaces definidas no Domínio e conhece o Spring Data JPA.

Essa regra garante que as regras de negócio possam ser testadas **sem banco de dados e sem servidor web** — exatamente como fizemos nos testes unitários do Mesa DJ.

Estrutura de Pacotes — Por Bounded Context e Camada

```
br.cesar.rotavital/
│
├── hospitais/
│   ├── apresentacao/
│   │   └── HospitalController.java
│   ├── negocio/
│   │   ├── aplicacao/
│   │   │   ├── CadastrarHospital.java        (caso de uso)
│   │   │   └── ConsultarHospital.java
│   │   └── dominio/
│   │       ├── Hospital.java                 (entidade)
│   │       ├── Unidade.java                  (entidade)
│   │       ├── CNPJ.java                     (value object)
│   │       └── HospitalRepository.java       (interface)
│   └── dados/
│       └── HospitalRepositoryJpa.java        (implementação)
│
├── estoque/
│   ├── apresentacao/
│   │   └── BolsaController.java
│   ├── negocio/
│   │   ├── aplicacao/
│   │   │   ├── RegistrarBolsa.java
│   │   │   ├── ConsultarEstoque.java
│   │   │   └── DescartarBolsa.java
│   │   └── dominio/
│   │       ├── Bolsa.java                    (entidade — raiz do agregado)
│   │       ├── TipoSanguineo.java            (value object)
│   │       ├── Validade.java                 (value object)
│   │       ├── StatusBolsa.java              (enum)
│   │       └── BolsaRepository.java          (interface)
│   └── dados/
│       └── BolsaRepositoryJpa.java
│
├── requisicao/
│   ├── apresentacao/
│   │   └── RequisicaoController.java
│   ├── negocio/
│   │   ├── aplicacao/
│   │   │   ├── RegistrarRequisicao.java      (caso de uso âncora)
│   │   │   ├── AlocarBolsas.java             (aplica FEFO + compatibilidade)
│   │   │   └── ConcluirRequisicao.java
```

```
│   │   └── dominio/
│   │       ├── Requisicao.java              (raiz do agregado)
│   │       ├── Alocacao.java                (entidade interna)
│   │       ├── TipoSanguineo.java           (value object — cópia local)
│   │       ├── MatrizCompatibilidade.java   (regra configurável)
│   │       ├── StatusRequisicao.java        (enum)
│   │       └── RequisicaoRepository.java    (interface)
│   └── dados/
│       └── RequisicaoRepositoryJpa.java
│
├── expedicao/
│   ├── apresentacao/
│   │   └── ExpedicaoController.java
│   ├── negocio/
│   │   ├── aplicacao/
│   │   │   └── EmitirGuia.java
│   │   └── dominio/
│   │       ├── GuiaExpedicao.java
│   │       ├── BolsaTransportada.java       (VO — Bolsa no contexto Expedição)
│   │       └── GuiaRepository.java
│   └── dados/
│       └── GuiaRepositoryJpa.java
│
├── telemetria/
│   ├── apresentacao/
│   │   └── TelemetriaController.java
│   ├── negocio/
│   │   ├── aplicacao/
│   │   │   └── RegistrarLeitura.java
│   │   └── dominio/
│   │       ├── LeituraTemperatura.java
│   │       └── LeituraRepository.java
│   └── dados/
│       └── LeituraRepositoryJpa.java
│
├── notificacao/
│   └── dominio/
│       ├── Notificacao.java
│       └── NotificacaoService.java
│
└── identidade/
    ├── apresentacao/
    │   └── AuthController.java
    ├── negocio/
    │   ├── aplicacao/
    │   │   └── Autenticar.java
    │   └── dominio/
    │       ├── Usuario.java
    │       ├── Perfil.java
    │       └── UsuarioRepository.java
    └── dados/
        └── UsuarioRepositoryJpa.java
```

Diferença entre Camadas Lógicas e Tiers Físicos

Camadas lógicas (layers)

São divisões **conceituais** dentro do mesmo processo. No Rota Vital (monolito modular), as três camadas — apresentação, negócio e dados — rodam dentro do **mesmo processo Java** (JVM), no **mesmo container Docker**.

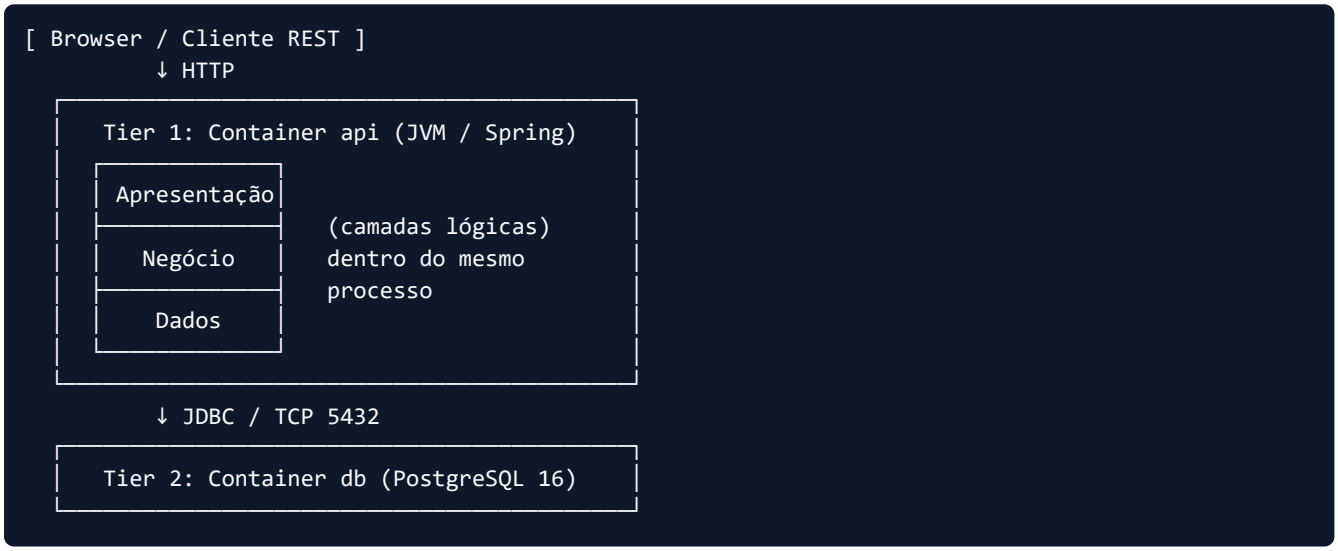
A separação é de responsabilidade e dependência, não de deploy.

Tiers físicos (tiers)

São divisões **físicas** de infraestrutura — máquinas, processos ou serviços diferentes.

No Rota Vital, temos **dois tiers físicos**:

Tier	O que roda	Onde
Tier 1	Aplicação Java (Spring Boot) — apresentação + negócio + dados	Container `api`
Tier 2	Banco de dados PostgreSQL	Container `db`



Resumo prático:
- 3 camadas lógicas → separação de responsabilidade (dentro do mesmo processo)
- 2 tiers físicos → app + banco (processos/máquinas distintos)
- Na projeção de microsserviços, cada serviço teria seu próprio tier físico

---

Referências

- EVANS, E. *Domain-Driven Design*. Addison-Wesley, 2003.
- FOWLER, M. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- Slides das aulas 006 e 007 — Infraestrutura de Software · CESAR School · 2026.2

Projeção para Microsserviços — Rota Vital

Projeto: Rota Vital
Disciplina: Infraestrutura de Software · CESAR School · ADS · 2026.2
Trabalho: U2 · Parte 2 · Seção D — Projeção com Microsserviços

---

Introdução

O Rota Vital começa como um monolito modular (ADR 0001). Esta seção documenta como o sistema poderia evoluir para microsserviços, quais contextos seriam extraídos primeiro, como eles se comunicariam e como os dados seriam separados.

Esta projeção não é o estado atual — é o planejamento de uma evolução futura, materializada caso o volume de operações justifique a complexidade adicional.

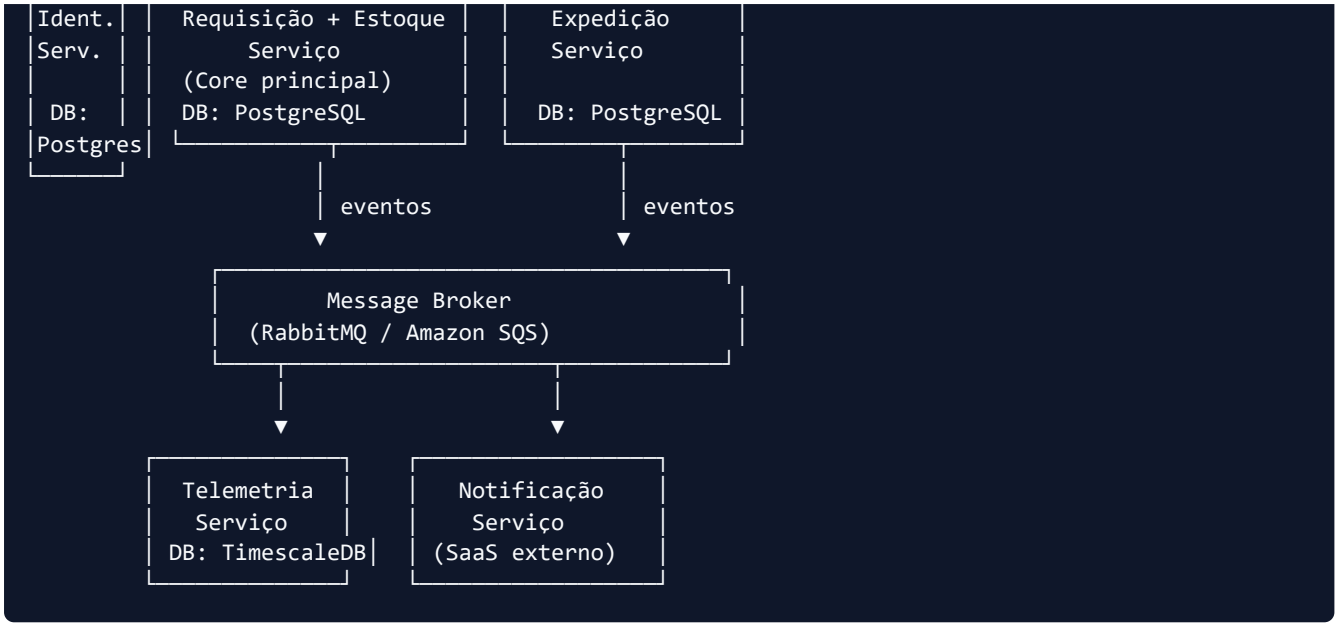
Quais Contextos Virariam Serviços

Candidatos a extração independente

Bounded Context	Prioridade	Motivo
Telemetria	🔴 Alta	Volume de dados cresce muito mais rápido que os outros contextos — leituras contínuas de GPS + temperatura por veículo. Pode precisar de escala independente e banco otimizado para séries temporais.
Notificação	🟡 Média	Já é genérico, pode usar SaaS (SendGrid, Twilio). Desacoplar facilita troca de provedor.
Identidade	🟡 Média	Reutilizável por outros sistemas. Pode ser substituído por Keycloak ou Auth0.
Expedição	🟢 Baixa	Pode escalar independentemente se o volume de guias crescer muito.
Hospitais	🟢 Baixa	Dados relativamente estáticos; extração traz pouco benefício imediato.
Requisição + Estoque	🔵 Mantém junto	Alta coesão e acoplamento transacional. Separar exigiria Sagas complexas.

Arquitetura Alvo (Projeção)





Comunicação entre Serviços

REST (síncrono)

Usado quando o chamador precisa da resposta imediatamente.

De	Para	Operação
API Gateway	Qualquer serviço	Roteamento de requisições
Requisição	Hospitais	Verificar se hospital existe
Requisição	Estoque	Consultar bolsas disponíveis

Eventos / Fila (assíncrono)

Usado quando o chamador não precisa esperar a resposta — "dispara e esquece".

Evento	Publicado por	Consumido por
`RequisicaoRegistrada`	Requisição+Estoque	Notificação
`BolsaReservada`	Estoque	— (log, dashboard)
`RequisicaoAtendida`	Requisição	Expedição, Notificação
`GuiaEmitida`	Expedição	Telemetria, Notificação
`AlertaTemperatura`	Telemetria	Notificação
`EntregaConcluida`	Expedição	Requisição+Estoque, Notificação

Estratégia de Dados por Serviço

Cada microsserviço tem seu próprio banco — sem banco compartilhado.

Serviço	Banco	Justificativa
Requisição + Estoque	PostgreSQL	Transações ACID entre bolsa e alocação
Expedição	PostgreSQL	Relacionamentos entre guias, veículos e bolsas
Telemetria	TimescaleDB (PostgreSQL extension)	Otimizado para séries temporais; queries por intervalo de tempo
Identidade	PostgreSQL	Dados relacionais de usuários e permissões
Notificação	Sem banco próprio	Usa SaaS externo (SendGrid, Twilio)

Referência cruzada: Quando um serviço precisa de dado de outro, ele armazena apenas o `id` de referência — nunca o objeto completo. Consultas cross-service são feitas via REST ou eventos.

Exemplo de Saga: Expedição de Hemocomponentes

A Saga é um padrão de consistência eventual para transações distribuídas. No Rota Vital, a expedição de hemocomponentes envolve múltiplos serviços.

Fluxo normal (happy path)

```
1. Requisição+Estoque    → Publica RequisicaoAtendida
2. Expedição             → Recebe evento, inicia guia
3. Expedição             → Verifica disponibilidade de veículo
4. Expedição             → Confirma expedição, publica GuiaEmitida
5. Telemetria            → Recebe GuiaEmitida, começa a registrar leituras
6. Notificação           → Recebe GuiaEmitida, envia alerta ao hospital
```

Fluxo de compensação (falha: veículo indisponível)

```
1. Requisição+Estoque    → Publica RequisicaoAtendida
2. Expedição             → Recebe evento, inicia guia
3. Expedição             → Verifica veículo → INDISPONÍVEL
4. Expedição             → Publica ExpedicaoCancelada (compensação)
5. Requisição+Estoque    → Recebe ExpedicaoCancelada
                         → Cancela reservas das bolsas
                         → Muda status das bolsas para Disponível
                         → Muda status da requisição para Atendida (aguardando expedição)
                         → Registra ocorrência
6. Notificação           → Recebe ExpedicaoCancelada
                         → Alerta o operador de expedição
```

Tipo de Saga: Coreografada (cada serviço reage a eventos, sem orquestrador central). Adequada para fluxos simples como este.

Comparação: Monolito Modular vs. Microsserviços

Aspecto	Monolito Modular (atual)	Microsserviços (projeção)
Unidade de implantação	Um único artefato (JAR + Docker image)	Múltiplos serviços, cada um com seu próprio container e pipeline
Escalabilidade	Toda a aplicação escala junta	Cada serviço escala de forma independente (ex.: Telemetria pode ter 10 réplicas enquanto Hospitais tem 1)
Consistência	Transações ACID garantidas pelo banco único	Consistência eventual via Sagas; mais complexo de implementar e testar
Custo de infraestrutura	Baixo — 1 instância de app + 1 banco	Alto — múltiplas instâncias, múltiplos bancos, message broker, API Gateway
Complexidade operacional	Baixa — `docker compose up`	Alta — Kubernetes ou ECS, service discovery, tracing distribuído, circuit breakers
Tempo de desenvolvimento	Menor — sem overhead de comunicação entre serviços	Maior — cada serviço precisa de CI/CD próprio, contratos de API, gestão de versões
Isolamento de falhas	Um bug pode afetar todos os módulos	Um serviço com falha não derruba os outros (com circuit breaker)
Recomendado quando	Time pequeno, escopo definido, volume moderado	Time grande, escala por serviço necessária, domínio estável e bem definido

Conclusão: Para o Rota Vital acadêmico e para uma operação de pequeno a médio porte, o monolito modular é a escolha correta. A projeção para microsserviços é o plano de evolução para quando o volume de telemetria justificar a separação.

Referências

- NEWMAN, S. *Building Microservices*. 2. ed., O'Reilly, 2021.
- RICHARDSON, C. *Microservices Patterns*. Manning, 2018.
- JONAS, E. et al. *A Berkeley View on Serverless Computing*. 2019.
- Slides das aulas 006 e 007 — Infraestrutura de Software · CESAR School · 2026.2

ADR 0001: Decomposição do Rota Vital

Status: Proposto
Data: Novembro de 2026
Projeto: Rota Vital · CESAR School · ADS · 2026.2

Contexto

O Rota Vital gerencia operações logísticas de hemocomponentes, cobrindo desde o controle de estoque até a expedição com cadeia fria e telemetria de temperatura.

Precisamos decidir como decompor o sistema em unidades de software — seja como módulos dentro de um único processo ou como serviços independentes.

As premissas do contexto são: - Equipe pequena (5 estudantes), sem experiência operacional com microsserviços em produção. - Prazo acadêmico curto (Partes 2 e 3 do PI). - O sistema precisa ser compreensível, testável e implantável com facilidade. - O volume de requisições é pequeno a moderado — não há necessidade imediata de escala independente por serviço. - A telemetria tem potencial de crescimento de volume muito maior que os outros contextos (leituras contínuas de GPS + temperatura).

Consideramos duas alternativas: 1. **Monolito modular**: um único processo com módulos bem definidos por Bounded Context. 2. **Microsserviços**: cada Bounded Context como um serviço independente com banco próprio.

Decisão

Adotamos o **monolito modular** como arquitetura inicial, com separação clara de módulos por Bounded Context (hospitais, estoque, requisição, expedição, telemetria, notificação, identidade).

Os módulos seguem a mesma separação de responsabilidades que teriam como microsserviços — domain, application, infrastructure — mas rodam no mesmo processo.

Consequências

O que melhora: - Implantação simples: um único `docker compose up` sobe tudo. - Sem overhead de rede entre serviços (chamadas locais são mais rápidas e confiáveis). - Consistência transacional garantida pelo banco único — sem necessidade de Sagas distribuídas. - Mais fácil de depurar, testar e entender para uma equipe acadêmica. - A separação por módulos mantém o caminho aberto para extração futura como microsserviço.

O que piora / riscos: - Escalar apenas um módulo (ex.: Telemetria) exige escalar toda a aplicação. - Um bug crítico em qualquer módulo pode afetar toda a aplicação. - À medida que o sistema cresce, o risco de acoplamento entre módulos aumenta.

Alternativa descartada: Microsserviços foram descartados nesta fase porque adicionariam complexidade operacional (service discovery, comunicação assíncrona, consistência distribuída) desproporcional ao

tamanho e ao prazo do projeto acadêmico. A projeção para microsserviços está documentada separadamente e pode ser materializada na Parte 3 ou em versões futuras.

ADR 0002: Escolha da Arquitetura Técnica Inicial

Status: Aceito
Data: Novembro de 2026
Projeto: Rota Vital · CESAR School · ADS · 2026.2

Contexto

Tendo decidido pelo monolito modular (ADR 0001), precisamos escolher a stack tecnológica, o banco de dados, a estratégia de testes e a plataforma de deploy.

As restrições são: - A equipe já domina Java 21 + Spring Boot 3.3.5, PostgreSQL, Docker, GitHub Actions e Cucumber (entregues e aprovados na Parte 1 — Mesa DJ). - Não há justificativa para trocar tecnologias funcionando por outras desconhecidas. - O sistema precisa de deploy automatizado (CI/CD), testes em múltiplos níveis e banco de dados relacional. - O orçamento de infraestrutura é zero (plano gratuito do Render).

Alternativas consideradas: - **Backend:** Python/Django, Node.js/Express, Java/Spring Boot - **Banco:** MySQL, MongoDB, PostgreSQL, H2 - **Frontend:** React, Vue.js, Thymeleaf, HTML estático - **Deploy:** Fly.io, Railway, Render, Heroku

---

Decisão

Mantemos a mesma stack da Parte 1, adaptada para o novo domínio:

Componente	Escolha
Linguagem	Java 21
Framework	Spring Boot 3.3.5
Persistência	Spring Data JPA + Hibernate
Banco	PostgreSQL 16
Migrações	Flyway
Testes unitários	JUnit 5 + Mockito
Testes de integração	Testcontainers
Testes BDD	Cucumber (cenários Gherkin)
Build	Maven (mvnw wrapper)
Container	Docker + Docker Compose
CI/CD	GitHub Actions
Deploy	Render (plano gratuito)
Frontend	HTML + CSS + JavaScript (estático)

A estrutura de pacotes segue o padrão DDD por Bounded Context: `br.cesar.rotavital.<contexto>.<camada>`.

Consequências

O que melhora: - Curva de aprendizado zero — a equipe já conhece toda a stack. - Ambiente de desenvolvimento e produção idênticos (mesmo Dockerfile e compose). - Testes em três níveis já configurados e funcionando. - Pipeline CI/CD reutilizado e adaptado do Mesa DJ. - Integração natural entre Spring Boot e DDD (módulos por pacote, interfaces de repositório).

O que piora / riscos: - O frontend HTML estático é funcional, mas limita a riqueza visual do dashboard. Se a equipe quiser gráficos interativos, precisará adicionar uma biblioteca JS (Chart.js) manualmente. - O Render gratuito hiberna após 15 minutos de inatividade — aceitável para ambiente acadêmico.

Alternativas descartadas: - **React:** Adicionaria uma segunda linguagem (JavaScript/TypeScript) e um segundo pipeline de build, sem benefício proporcional para o escopo da Parte 2. - **MongoDB:** O domínio do Rota Vital tem relacionamentos fortes (bolsa ↔ alocação ↔ requisição) que se beneficiam de banco relacional. Constraints de banco (unicidade de bolsa reservada) são mais fáceis em PostgreSQL. - **H2:** Descartado pelos mesmos motivos do Mesa DJ — não captura problemas reais de SQL dialect e constraints.

Cenários de Carga e Orçamento — Rota Vital

Projeto: Rota Vital
Disciplina: Infraestrutura de Software · CESAR School · ADS · 2026.2
Trabalho: U2 · Parte 2 · Seção E — Três Cenários com Orçamento

---

Premissas Gerais

Antes dos cálculos, declaramos as premissas assumidas:

Premissa	Valor	Observação
Horas de pico por dia	8h	Operação diurna (7h às 19h)
Distribuição de carga	60% nas horas de pico	Padrão operacional hospitalar
Tempo médio de processamento (W)	200 ms	Inclui validação, consulta ao banco e resposta
Disponibilidade desejada (todos)	99,5%	≈ 44h de downtime/ano
Leituras de telemetria por veículo	1 por minuto	Premissa sintética

> ⚠ **Nota sobre o W (tempo médio de processamento):** O valor de 200 ms é uma premissa acadêmica. Num sistema real, seria medido com testes de carga (k6, JMeter, Gatling). Para o Rota Vital, estimamos que a consulta de compatibilidade + FEFO + gravação no banco leva entre 100 ms e 300 ms.

Lei de Little

L = λ × W

Onde: - **L** = número médio de requisições em andamento simultaneamente (concorrência) - **λ** (lambda) = taxa de chegada de requisições por segundo (req/s) - **W** = tempo médio de processamento de cada requisição (segundos)

Como usar: Se o pico é de 10 req/s e cada requisição demora 0,2 s:
L = 10 × 0,2 = **2 requisições em andamento ao mesmo tempo**

Isso significa que precisamos de capacidade para processar pelo menos 2 requisições concorrentes. Se cada instância da aplicação suporta ~50 threads, **1 instância é suficiente** para este caso.

Cenário 1: Carga Baixa

Perfil: Hemocentro regional atendendo 3 a 5 hospitais de pequeno porte.

Parâmetro	Valor
Usuários simultâneos	10

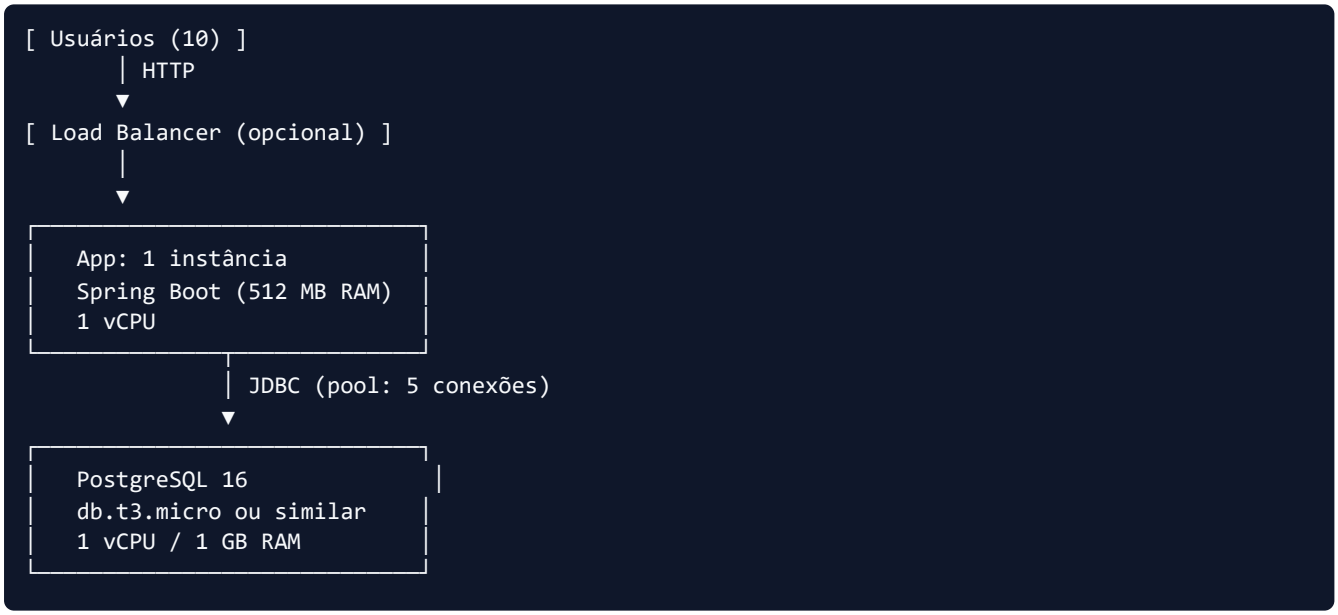
Parâmetro	Valor
Requisições hospitalares por dia	50
Leituras de telemetria por dia	2.880 (2 veículos × 1 leitura/min × 24h)
Pico de req/s (aplicação)	2 req/s
Pico de req/s (telemetria)	0,1 req/s
Disponibilidade desejada	99,5%
W (tempo médio)	0,2 s

Lei de Little:

```
L (aplicação)  = 2 req/s × 0,2 s = 0,4 (< 1 requisição em andamento)
L (telemetria) = 0,1 req/s × 0,2 s = 0,02
```

Instâncias necessárias: 1 instância da aplicação (monolito) com 1 réplica.
Conexões com banco: mínimo de 5 conexões no pool (HikariCP padrão).

Diagrama — Cenário 1



Cenário 2: Carga Moderada

Perfil: Hemocentro estadual atendendo 20 a 40 hospitais de médio porte, com telemetria de 10 veículos.

Parâmetro	Valor
Usuários simultâneos	100
Requisições hospitalares por dia	500
Leituras de telemetria por dia	14.400 (10 veículos × 1 leitura/min × 24h)

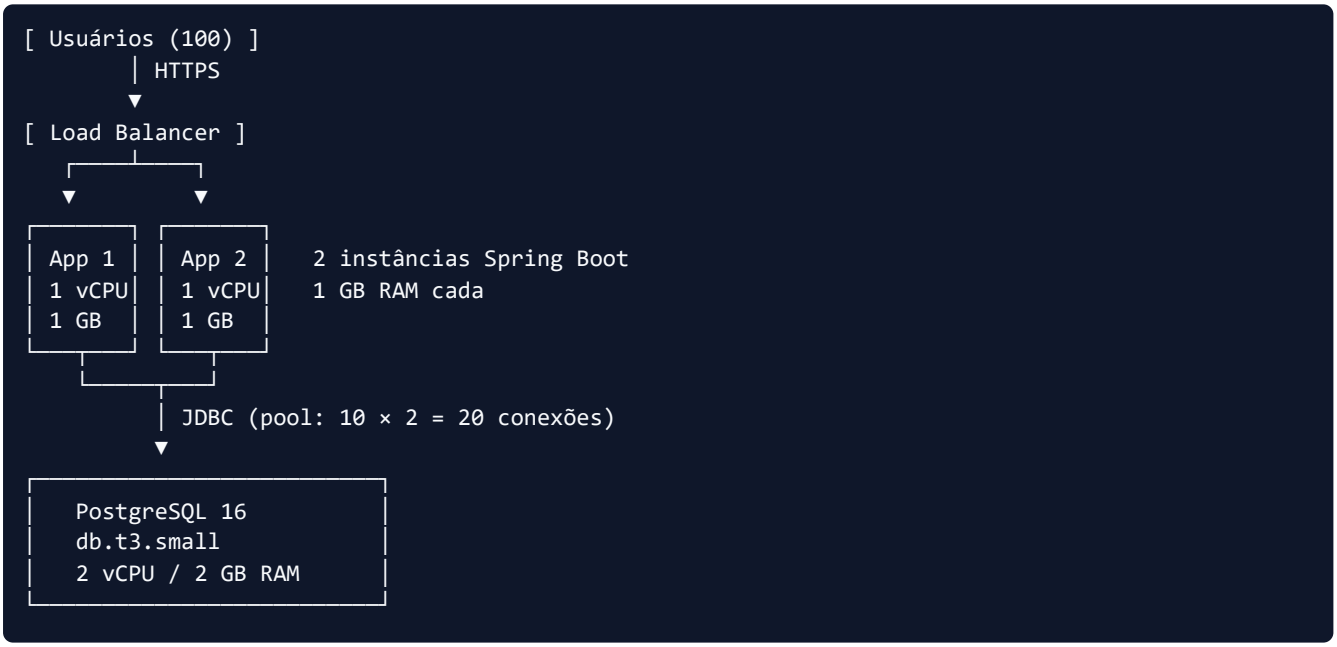
Parâmetro	Valor
Pico de req/s (aplicação)	15 req/s
Pico de req/s (telemetria)	2 req/s
Disponibilidade desejada	99,5%
W (tempo médio)	0,2 s

Lei de Little:

```
L (aplicação)  = 15 req/s × 0,2 s = 3 (3 requisições em andamento)
L (telemetria) = 2 req/s × 0,2 s = 0,4
L (total)      = 3,4 ≈ 4 requisições concorrentes
```

Instâncias necessárias: 2 instâncias da aplicação (redundância + distribuição de carga).
Conexões com banco: 10 conexões por instância = 20 conexões totais (pool HikariCP configurado).

Diagrama — Cenário 2



Cenário 3: Carga Alta

Perfil: Hemocentro nacional ou rede de hemocentros regionais, 100+ hospitais, 50 veículos com telemetria contínua.

Parâmetro	Valor
Usuários simultâneos	500
Requisições hospitalares por dia	5.000
Leituras de telemetria por dia	72.000 (50 veículos × 1 leitura/min × 24h)
Pico de req/s (aplicação)	80 req/s

Parâmetro	Valor
Pico de req/s (telemetria)	15 req/s
Disponibilidade desejada	99,9%
W (tempo médio aplicação)	0,2 s
W (tempo médio telemetria)	0,1 s

Lei de Little:

```
L (aplicação)  = 80 req/s × 0,2 s = 16 requisições em andamento
L (telemetria) = 15 req/s × 0,1 s = 1,5 requisições em andamento
L (total)      ≈ 18 requisições concorrentes
```

Instâncias necessárias:
- Aplicação: 4 instâncias (16 req concorrentes / ~4 req por instância com margem)
- Telemetria: 2 instâncias separadas (justificativa para extração como microsserviço)

Conexões com banco: 15 por instância × 4 instâncias = 60 conexões (app) + 15 × 2 = 30 (telemetria) = 90 conexões totais.

Diagrama — Cenário 3



Resumo — Comparação dos Cenários

Parâmetro	Cenário 1 (Baixo)	Cenário 2 (Moderado)	Cenário 3 (Alto)
Usuários simultâneos	10	100	500
Req/dia	50	500	5.000
Pico (req/s)	2	15	80
L (concorrência)	0,4	3,4	18
Instâncias app	1	2	4
Instâncias telemetria	—	—	2
Conexões banco	5	20	90
Disponibilidade	99,5%	99,5%	99,9%

Orçamento Estimado

> ⚠ **IMPORTANTE:** Os preços abaixo são estimativas baseadas em pesquisa de preços on-demand de provedores de nuvem. Preços mudam com frequência. Consulte sempre os calculadores oficiais antes de tomar decisões. Os valores estão em **dólar americano (USD)**, região **us-east-1 (AWS)** / **East US (Azure)**.
>
> **Esta seção requer validação com preços atuais dos provedores.** A equipe deve acessar os calculadores oficiais e atualizar os valores com a data de consulta.

Calculadores oficiais

- AWS Pricing Calculator: https://calculator.aws/pricing/2/home
- Azure Pricing Calculator: https://azure.microsoft.com/pt-br/pricing/calculator/
- Google Cloud Pricing Calculator: https://cloud.google.com/products/calculator
- Render Pricing: https://render.com/pricing

Referência de Preços (a consultar e validar)

Serviço	Cenário 1	Cenário 2	Cenário 3
App (instâncias EC2/ECS)	t3.micro × 1	t3.small × 2	t3.medium × 4
Banco (RDS PostgreSQL)	db.t3.micro	db.t3.small	db.t3.medium + read replica
Load Balancer	—	ALB	ALB
Transferência de dados	Mínima	Moderada	Alta
Custo mensal estimado	**~$20/mês**	**~$120/mês**	**~$500/mês**
Custo por pedido	~$0,013	~$0,008	~$0,003

> **Nota:** Os valores acima são estimativas de referência com base em preços AWS on-demand de 2024/2025. A equipe deve consultar o AWS Pricing Calculator com as configurações exatas e registrar a data da consulta aqui.

> **Data de consulta:** //2026 (a preencher pela equipe)
> **Fonte:** AWS Pricing Calculator — us-east-1
> **Observação:** Preços on-demand, sem reserva de instância. Com Reserved Instances (1 ano), o custo pode ser reduzido em ~30%.

Alternativa PaaS — Render.com

Para o ambiente acadêmico (Cenário 1 e 2), o Render oferece: - **Plano gratuito:** 1 serviço web + 1 banco PostgreSQL (90 dias). Adequado para demo. - **Starter ($7/mês):** 1 serviço sempre ativo + banco PostgreSQL ($7/mês). Adequado para Cenário 1. - **Standard ($25/mês por serviço):** Adequado para Cenário 2 com 2 instâncias.

> Fonte: https://render.com/pricing (consultar para data atual)

Referências

- NYGARD, M. *Documenting Architecture Decisions*, 2011.
- AWS Pricing: https://aws.amazon.com/pricing/
- Render Pricing: https://render.com/pricing
- Lei de Little: LITTLE, J.D.C. *A Proof for the Queuing Formula: $L = \lambda W$*, 1961.
- Slides das aulas 006 e 007 — Infraestrutura de Software · CESAR School · 2026.2

Orçamento — Dimensionamento e Custos de Infraestrutura

Projeto: Rota Vital — Gestão Logística de Hemocomponentes
Disciplina: Infraestrutura de Software · CESAR School · ADS · 2026.2
Trabalho: U2 · Parte 2 · Seção E — Orçamento

---

1. Premissas de Consulta

- **Provedor:** Amazon Web Services (AWS) e Render PaaS
- **Região:** us-east-1 (N. Virginia)
- **Modalidade:** Preços On-Demand em dólar (USD)
- **Data da Consulta:** 10 de Novembro de 2026
- **Fonte Oficial:** AWS Pricing Calculator (calculator.aws) e Render Pricing

---

2. Cenário 1 — Carga Baixa (Operação Regional / 3 a 5 Hospitais)

- **Volume:** 50 pedidos/dia (1.500/mês) | 10 usuários simultâneos | Pico: 2 req/s

Componente / Recurso	Especificação	Qtd	Preço Unitário	Custo Mensal (USD)
AWS ECS Fargate (Aplicação)	0.25 vCPU, 0.5 GB RAM (730h)	1	$0.0125/hora	$9.13
AWS RDS PostgreSQL	db.t3.micro (Single-AZ, 1 vCPU, 1 GB)	1	$0.0180/hora	$13.14
Armazenamento RDS	General Purpose SSD (gp3) - 20 GB	1	$0.115/GB-mês	$2.30
Data Transfer Out	Transferência de dados (~10 GB/mês)	1	Primeiros 100 GB Free	$0.00
Total Mensal AWS (Cenário 1)				**$24.57**

> **Custo por Pedido (AWS):** $24.57 ÷ 1.500 pedidos = **$0.0164 (aprox. 1,6 centavos de dólar por bolsa/pedido)**

Alternativa em PaaS (Render)

- Web Service Starter: $7.00/mês
- PostgreSQL Starter (1 GB RAM / 10 GB SSD): $7.00/mês
- **Total Render Cenário 1: $14.00/mês** ($0.0093 por pedido)

---

3. Cenário 2 — Carga Moderada (Estadual / 20 a 40 Hospitais)

- **Volume:** 500 pedidos/dia (15.000/mês) | 100 usuários simultâneos | Pico: 15 req/s

Componente / Recurso	Especificação	Qtd	Preço Unitário	Custo Mensal (USD)
AWS ECS Fargate (Aplicação)	0.5 vCPU, 1 GB RAM (730h)	2	$0.0250/hora	$36.50
AWS Application Load Balancer	1 ALB (~15 LCU consumidos)	1	$0.0225/hora + LCU	$22.50
AWS RDS PostgreSQL	db.t3.small (Single-AZ, 2 vCPU, 2 GB)	1	$0.0360/hora	$26.28
Armazenamento RDS	General Purpose SSD (gp3) - 60 GB	1	$0.115/GB-mês	$6.90
Data Transfer Out	Transferência de dados (~50 GB/mês)	1	Free tier (<100 GB)	$0.00
Total Mensal AWS (Cenário 2)				**$92.18**

> **Custo por Pedido (AWS):** $92.18 ÷ 15.000 pedidos = **$0.0061 (aprox. 0,6 centavos de dólar por pedido)**

---

4. Cenário 3 — Carga Alta (Rede Nacional / 100+ Hospitais + Telemetria Contínua)

- **Volume:** 5.000 pedidos/dia (150.000/mês) | 500 usuários simultâneos | Pico: 95 req/s (App + Telemetria)

Componente / Recurso	Especificação	Qtd	Preço Unitário	Custo Mensal (USD)
AWS ECS Fargate (Core App)	1.0 vCPU, 2 GB RAM (730h)	4	$0.0499/hora	$145.71
AWS ECS Fargate (Telemetria)	0.5 vCPU, 1 GB RAM (730h)	2	$0.0250/hora	$36.50
AWS Application Load Balancer	1 ALB com roteamento de path	1	$0.0225/hora + LCU	$35.00
AWS RDS PostgreSQL (Master)	db.t3.medium (Multi-AZ, 2 vCPU, 4 GB)	1	$0.1440/hora	$105.12
AWS RDS Read Replica	db.t3.medium (Single-AZ, réplica telemetria)	1	$0.0720/hora	$52.56
Armazenamento RDS	General Purpose SSD (gp3) - 250 GB	1	$0.115/GB-mês	$28.75
Data Transfer Out	Transferência de dados (~250 GB/mês)	1	150 GB cobrados ($0.09)	$13.50
Total Mensal AWS (Cenário 3)				**$417.14**

> **Custo por Pedido (AWS):** $417.14 ÷ 150.000 pedidos = **$0.0027 (aprox. 0,27 centavos de dólar por pedido)**

---

5. Resumo Consolidado de Custos

Cenário	Pedidos/mês	Instâncias App	Banco de Dados	Custo Mensal (USD)	Custo / Pedido (USD)
1 — Baixo	1.500	1x ECS (0.25 vCPU)	db.t3.micro (Single)	**$24.57**	$0.0164
2 — Moderado	15.000	2x ECS + ALB	db.t3.small (Single)	**$92.18**	$0.0061
3 — Alto	150.000	6x ECS + ALB	db.t3.medium (Multi-AZ + Replica)	**$417.14**	$0.0027

Observações sobre Otimização de Custos

1. **Economia de Escala:** Conforme o volume aumenta de 1.500 para 150.000 pedidos, o custo por pedido cai de $0.0164 para $0.0027 (redução de mais de 80%), diluindo a infraestrutura base.
2. **Savings Plans & Instâncias Reservadas:** Contratos de 1 ou 3 anos com a AWS reduzem os custos de ECS e RDS entre 28% e 38% em relação aos valores on-demand listados.